Software Requirements Specification — PharmaNet

Version 2.0 | June 2026

> 📍 **This document is part of the pharmanet-org/SRS documentation repository.**
> 🌐 **Live deployments: Admin Portal · Help Center · Mobile APK**
> 📦 **Source code: github.com/pharmanet-org**

---

1. Project Description

1.1 Platform Overview

PharmaNet is a multi-platform verified pharmacies network and search marketplace that connects customers with licensed pharmacies. It enables customers to browse medications, find nearby pharmacies, place orders, track deliveries, and communicate with pharmacists — while providing pharmacy owners and platform administrators with full operational control.

1.2 Problem Statement

Patients in Ethiopia face significant challenges accessing genuine medications from verified pharmacies. Existing solutions lack:
- A centralized directory of licensed pharmacies
- Real-time medication availability checking
- Transparent pricing and prescription handling
- Integrated payment processing in local currency (ETB)
- Multi-language support (English + Amharic)

PharmaNet addresses these gaps by providing a unified platform with verified pharmacy listings, real-time inventory, prescription management, and Chapa payment integration.

1.3 Target Users

Role	Description	Platform Access
Customer	End users browsing, ordering medications, chatting with pharmacies	Mobile App (Android, iOS, Web)
Seller	Pharmacy owners managing inventory, processing orders, promotions	Seller Web Portal + Mobile App
Admin	Platform administrators managing users, approvals, content, reports	Admin Web Portal

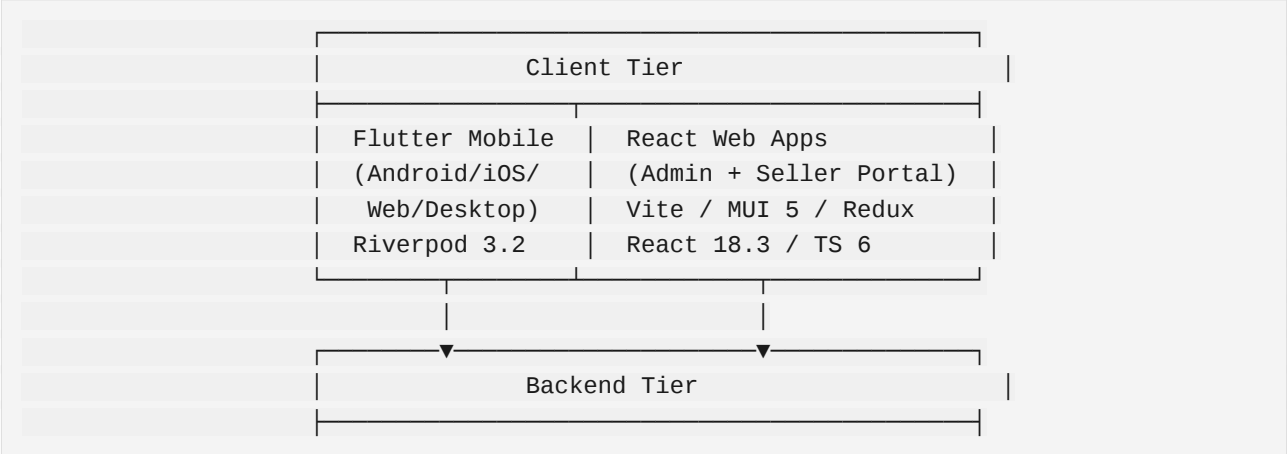
1.4 Project Scope

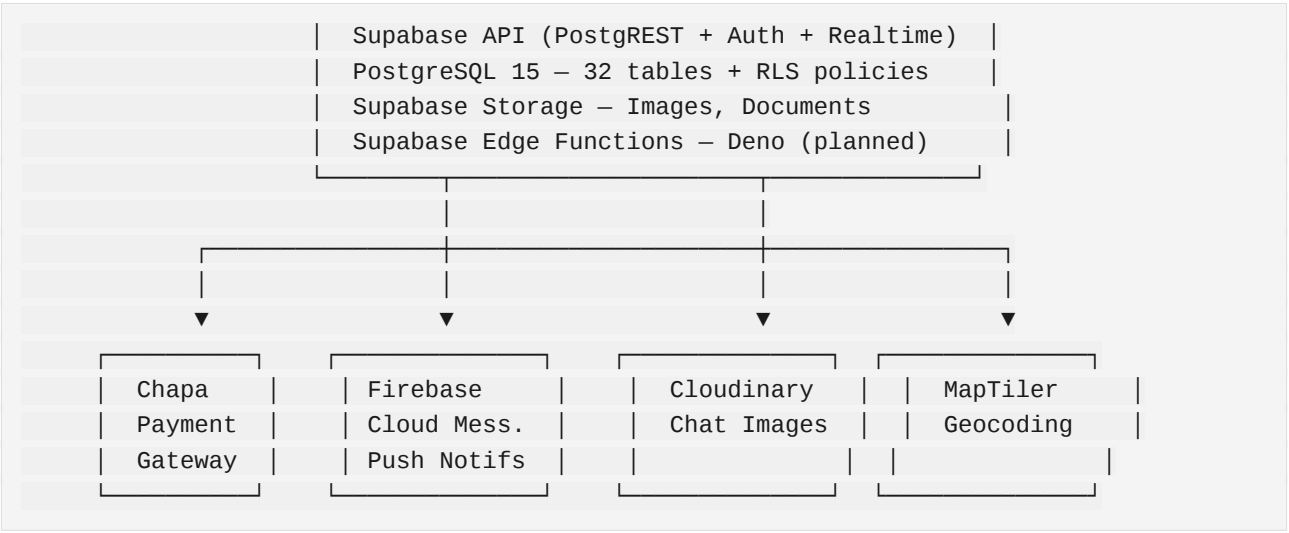
- **In Scope**: Pharmacy directory, product catalog with variants, cart & checkout, order management with real-time tracking, Chapa payment gateway, customer-pharmacist chat, promotions & offers, pharmacy subscriptions, featured product boosting, CMS (banners, static pages), push notifications, admin user/product/order management, bilingual support (EN + AM).
- **Out of Scope**: Prescription fulfillment/delivery logistics, insurance billing, telemedicine consultations, multi-language beyond English/Amharic.

---

2. System Design / Architecture

2.1 High-Level Architecture





2.2 Flutter Mobile App Architecture

Clean Architecture with feature-first organization:

```
lib/
├── main.dart                          # Entry, Firebase + Supabase init
├── core/
│   ├── api/                           # 20 API service classes (Supabase wrappers)
│   ├── config/                        # App + payment configuration
│   ├── constants/                     # Colors, strings, app constants
│   ├── enums/                         # UserRole enum
│   ├── models/                        # 14 Freezed data models
│   ├── providers/                     # 25 Riverpod state providers
│   ├── routes/                        # Named routes
│   ├── services/                      # 12 service classes
│   ├── theme/                         # AppTheme (light/dark)
│   └── utils/                         # Validators, extensions, logger
├── features/
│   ├── auth/                          # Login, register, OTP, password reset
│   ├── cart/                          # Cart management, checkout flow
│   ├── onboarding/                    # First-time user introduction
│   ├── orders/                        # Order list, detail, tracking timeline
│   ├── pharmacist/                    # Seller dashboard, inventory, orders
│   ├── pharmacy/                      # Catalog, detail, map, registration
│   ├── profile/                       # Profile, addresses, security, settings
│   └── public/                        # Home, notifications, wishlist, search
├── widgets/                           # Reusable UI components
└── l10n/                              # English + Amharic locales
```

2.3 React Web App Architecture (Admin + Seller)

```
src/
├── main.jsx                           # Entry point
```

```
├── App.jsx                              # Root with Supabase auth + MUI theme
├── routes.jsx                           # Lazy-loaded route config
├── assets/styles/                       # MUI themes (light/dark)
├── components/                          # Reusable UI components
│   ├── common/                          # DataTable, modals, spinners, error boundary
│   ├── layouts/                         # Header, Sidebar, MainLayout, AuthLayout
│   ├── dashboard/                       # Stats cards, sales/revenue/performance
charts
│   ├── products/                        # Product form, list, variants, approval
│   ├── orders/                          # Order list, timeline, status updates
│   ├── users/                           # User management, pharmacy approval
│   ├── payments/                        # Transactions, Chapa config
│   ├── cms/                             # Banners, static pages
│   ├── reports/                         # Sales, performance, export
│   └── settings/                        # General, payment, email, system
├── pages/                               # Lazy-loaded page components
├── services/                            # 18 Supabase service modules
├── store/                               # 9 Redux Toolkit slices
├── hooks/                               # useAuth, useDebounce, useExport, etc.
└── utils/                               # Constants, formatters, validators
```

2.4 Database Schema

32 tables across 8 domains:

Domain	Tables
Users	`profiles`, `sellers`, `addresses`
Catalog	`categories`, `brands`, `products`, `variant_options`, `variant_values`, `variant_combinations`, `variant_combination_values`
Commerce	`carts`, `cart_items`, `orders`, `order_items`, `payments`, `order_status_history`
Engagement	`reviews`, `wishlists`, `wishlist_items`, `chats`, `chat_participants`, `messages`, `notifications`
CMS	`banners`, `static_pages`, `settings`
Promotions	`promotions`, `offer_products`
Monetization	`featured_product_payments`, `pharmacy_subscriptions`, `featured_product_access`

Domain	Tables
Analytics	`analytics`

All tables protected by **Row-Level Security (RLS)** policies. Real-time enabled on `promotions` and `offer_products`.

2.5 Key Design Decisions

Decision	Rationale
Supabase over custom backend	Built-in auth, real-time, storage, RLS — near-zero backend code
Riverpod over BLoC	Simpler syntax, better testability, compile-time safety
Redux Toolkit over Context	Predictable state, middleware for real-time, DevTools
Freezed models	Immutable classes, JSON serialization, union types
Prisma schema	Type-safe client, auto-generated types, migration management
Chapa for payments	Ethiopian market focus, hosted checkout, ETB currency
MUI 5	Mature component library, responsive, customizable theme

---

3. API Documentation

3.1 Supabase API

All apps connect to a shared Supabase instance (`egullnxmzmkbhtksjglu.supabase.co`):

Service	Protocol	Purpose
PostgREST	REST over HTTP	CRUD on all 32 tables
Auth	REST + JWT	Sign-up, login, password reset, session

Service	Protocol	Purpose
(GoTrue)		
Realtime	WebSocket	Live order/chat/promotion updates
Storage	REST + S3	Product images, pharmacy docs, banners

3.2 Mobile App API Layer (20 classes)

Class	Key Methods
`AuthApi`	signUp, signIn, signOut, resetPassword, updatePassword
`ProductApi`	fetchProducts, searchProducts, getProductById, getFeaturedProducts
`OrderApi`	createOrder, getOrders, getOrderById, updateOrderStatus
`CartApi`	getCart, addItem, removeItem, updateQuantity, clearCart
`PharmacyApi`	getNearbyPharmacies, getPharmacyDetail, getPharmacyProducts
`CategoryApi`	getCategories, getCategoryTree
`ReviewApi`	getReviews, createReview, getRatingSummary
`ChatApi`	getChats, sendMessage, getMessages, createChat
`NotificationApi`	getNotifications, markAsRead, registerFCMToken
`PromotionApi`	getActivePromotions, claimOffer
`WishlistApi`	getWishlist, addItem, removeItem
`AddressApi`	getAddresses, addAddress, updateAddress, setDefault
`BrandApi`	getBrands
`ChatbotApi`	queryChatbot

Class	Key Methods
`CmsApi`	getBanners, getStaticPages
`FeaturedProductApi`	getFeaturedProducts
`OfferApi`	getOffers, createOffer
`ProfileApi`	getProfile, updateProfile, uploadAvatar
`PublicUserApi`	getPublicProfile
`SecuritySettingsApi`	getSettings, updatePasscode, updateBiometric

3.3 Web App Service Layer (18 modules)

Service	Purpose
`api.js`	Base API client (Axios)
`auth.js`	Login, logout, password reset, session
`supabase.js`	Supabase client initialization
`users.js`	User CRUD, role management, pharmacy approval
`products.js`	Product CRUD, image management, approval
`categories.js`	Hierarchical category management
`orders.js`	Order tracking, status management
`payments.js`	Transaction history, payment verification
`chapa.js`	Chapa payment gateway integration
`dashboard.js`	Dashboard stats, trends
`reports.js`	Sales/product performance analytics

Service	Purpose
`cms.js`	Banners, static pages management
`settings.js`	System-wide configuration
`notifications.js`	Notification management
`promotions.js`	Promotion CRUD, calendar
`offers.js`	Seller offer management
`featuredProducts.js`	Featured product payments
`pharmacySubscription.js`	Subscription plan management

3.4 External Integrations

Service	Integration Point	Auth Method
Chapa	Hosted checkout via `https://api.chapa.co/v1/transaction/initialize`	API Key (test: `CHASECK_TEST-*`)
Firebase Cloud Messaging	`firebase_messaging` SDK to FCM token stored in `profiles.fcm_token`	Firebase project credentials
Cloudinary	HTTP upload to `https://api.cloudinary.com/v1_1/{cloud}/image/upload`	API Key + Secret
MapTiler	Geocoding API for address autocomplete	API Key

---

4. Features & Functionalities

4.1 Mobile App (Customer-Facing)

Feature	Description
Authentication	Email/password sign-up, login, OTP, password reset, session management
Onboarding	First-time user introduction screens
Home Screen	Browse featured products, categories, nearby pharmacies, banners
Product Discovery	Search by name/category/brand, filter by price/rating/prescription, sort, variant selection (pack size, dosage)
Pharmacy Directory	Map-based pharmacy finder with geolocation, pharmacy profiles, ratings, contact info
Cart & Checkout	Multi-pharmacy cart, quantity management, address selection, prescription upload
Order Management	Place orders, real-time status tracking (pending to confirmed to processing to shipped to delivered), order history, cancellation
Payments	Chapa hosted checkout, payment status, transaction history
Promotions & Offers	View active promotions, claim seller discounts, auto-apply to cart
Notifications	Push notifications (FCM), in-app notification center, order updates
Profile & Security	Edit profile, app lock (PIN/biometric), change password, delete account, privacy settings
Wishlist	Save products, add/remove, view wishlist
Chat & Messenger	Real-time chat with pharmacy staff, image sharing via Cloudinary
Reviews & Ratings	Rate products (1-5 stars), write reviews, view aggregated ratings

Feature	Description
Pharmacist Dashboard	Seller-specific: order management, inventory, analytics, profile editing
Featured Products	View boosted/sponsored products labeled as "Ad"
Pharmacy Subscriptions	Subscribe to pharmacy plans for premium features
Localization	Full English + Amharic language support
Guest Access	Browse without login, prompted to authenticate at checkout

4.2 Admin Web Portal

Feature	Description
Dashboard	Platform-wide KPIs, revenue charts, user growth, order volume
User Management	View/manage customers and sellers, approve/reject pharmacy registrations, suspend users
Product Management	Review and approve/reject seller products, manage categories and brands
Order Management	View all platform orders, intervene in disputes, full lifecycle tracking
Payment Management	Monitor transactions, verify Chapa payments, process refunds
Content Management (CMS)	Manage homepage banners, static pages (About, Terms, Privacy)
Promotions	Create platform-wide promotions, approve/reject seller offers, promotion calendar
Featured Products	Oversee all boosted products, manage payment records
Pharmacy Subscriptions	Manage plans, oversee seller subscriptions, billing

Feature	Description
Reports & Analytics	Platform-wide reports, export to PDF/Excel, sales/product analytics
Settings	System configuration, notification templates, platform branding

4.3 Seller Web Portal

Feature	Description
Dashboard	Sales overview, revenue, order stats, recent activity
Product Management	CRUD, categories, brands, variants (SKU/pack/dosage), stock, images
Order Management	Incoming orders, status updates, fulfillment tracking
Payment Management	Transaction history, Chapa integration, payout tracking
Promotions & Offers	Time-limited discounts, offer management, promotion calendar
Featured Products	Boost products via paid promotion, payment flow
Pharmacy Subscriptions	Subscribe to plans, manage billing, payment callbacks
Reports & Analytics	Sales reports, product performance, export to PDF/Excel
Settings	Pharmacy profile, hours, shipping zones, notification prefs

---

5. Deployment Guide

5.1 Flutter Mobile App

```
cd pharmanet
```

```
# Android APK
flutter build apk --release
# Android App Bundle (Play Store)
flutter build appbundle --release

# iOS
flutter build ios --release
# Open in Xcode for archive:
open ios/Runner.xcworkspace

# Web (Vercel)
flutter build web
npx vercel --prod
```

5.2 React Web Apps (Admin + Seller)

```
# Admin Portal
cd pharmanet-admin
npm install
npx prisma generate
npm run build          # Output: dist/
npx vercel --prod

# Seller Portal
cd pharmanet-web
npm install
npx prisma generate
npm run build
npx vercel --prod
```

5.3 Chatbot

```
cd chatbot

# Docker
docker compose up --build

# Or manually
pip install -r requirements.txt
uvicorn src.main:app --host 0.0.0.0 --port 8000
```

5.4 Documentation Sites

```
# Help Center (Mintlify)
cd pharmanet-guide-docs
npm i -g mint
```

```
mint dev              # Preview at localhost:3000
# Deployed automatically via Vercel on push

# Technical Docs (Zensical)
cd docs
zensical build        # Output: site/
zensical serve        # Preview
```

5.5 Environment Variables (Production)

Variable	App	Source
`VITE_SUPABASE_URL`	Web apps	Supabase project
`VITE_SUPABASE_ANON_KEY`	Web apps	Supabase project
`DATABASE_URL`	Web apps (Prisma)	Supabase project
`VITE_CHAPA_PUBLIC_KEY`	Web apps	Chapa dashboard
`VITE_CHAPA_SECRET_KEY`	Web apps	Chapa dashboard

5.6 CI/CD

GitHub Actions workflows configured in each repository:

- **Mobile**: Flutter build + test on PR, release on tag
- **Web apps**: Vite build + Vitest on PR, deploy to Vercel on push to master
- **Dependabot**: Automated dependency PRs for GitHub Actions

---

6. User & Technical Documentation

6.1 Help Center — `pharmanet-guide-docs/`

Built with **Mintlify** (MDX + YAML frontmatter). Public site deployed on Vercel.

Section	Pages	Languages
Customer Guide	7 pages (create account, find products, visit pharmacies, place order, track orders, manage account)	English + Amharic
Seller Guide	10 pages (register, dashboard, products, orders, chat, reports, payments, promotions, settings)	English + Amharic
Help & Support	2 pages (FAQ, contact)	English + Amharic

6.2 Technical Docs — `docs/`

Built with **Zensical** (Markdown + TOML). 70+ pages covering:

Section	Pages
Platform Overview	5 pages (about, architecture, tech stack, database schema)
Mobile App (Flutter)	30+ pages (getting started, architecture, 16 features, 9 models, 8 providers)
Admin Web App	13+ pages (getting started, architecture, 12 features, services)
Seller Web App	10+ pages (getting started, architecture, 10 features, services)
Supabase Backend	16+ pages (schema, 14 tables, views, RLS, realtime, migrations)
Developer Guides	6 pages (local setup, database setup, contributing, debugging, deployment)

6.3 Project READMEs

Each project has a comprehensive README covering tech stack with pinned versions, folder structure, features, allowed users, hardcoded test credentials, prerequisites, run steps, configuration, and license.

Appendix A: Tech Stack Summary

Component	Technology	Version
Mobile Framework	Flutter	>3.41.0
Mobile Language	Dart	^3.9.2
Mobile State Mgmt	Riverpod	^3.2.1
Web Framework	React	^18.3.1
Web Build	Vite	^7.3.1
Web UI	MUI	^5.18.0
Web State	Redux Toolkit	^2.2.7
Database	PostgreSQL	15.x
Backend Service	Supabase	Latest
Payment Gateway	Chapa	^1.0.5
Push Notifications	Firebase Cloud Messaging	^16.2.0
Documentation	Mintlify / Zensical	Latest

Appendix B: Test Credentials

Role	Email	Password
Default Admin	`admin@PharmaNet.com`	`admin123`
Default Seller	`seller@pharmanet.co`	`seller123`

Role	Email	Password
	`m`	
Seed Admin	`admin@test.com`	(set in seed)
Seed Seller	`bole_pharma@test.com`	(set in seed)
Seed Customer	`abebe@test.com`	(set in seed)

> ⚠️ **These are test/dev credentials. Rotate all secrets before production.**
